

SIMATS ENGINEERING

ASSESSMENT TOOL 1

K. Lokesh Chandra
192425358

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

I K. Lokesh Chandra (192425358) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Lokesh Chandra

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.

Assume the relations:

Employee(EmpID, Name, DeptID, Salary)

Department(DeptID, DeptName)

Selection (σ)

Retrieves employees whose salary is greater than 50000.

σ Salary > 50000 (Employee)

Projection (π)

Displays only employee name and salary.

π Name, Salary (Employee)

Union ($\cup$)

Combines records from two employee tables.

Employee1 $\cup$ Employee2

Difference ($-$)

Displays employees who are not retired.

Employee – RetiredEmployee

Cartesian Product ($\times$)

Combines every employee with every department.

Employee $\times$ Department

Join ($\bowtie$)

Combines employee and department details using DeptID.

Employee $\bowtie$ Employee.DeptID = Department.DeptID Department

2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

The following SQL statements create the Department and Student tables with Primary Key and Foreign Key constraints.

```
CREATE TABLE DEPARTMENT(  
  DeptID INT PRIMARY KEY,  
  DeptName VARCHAR(30) NOT NULL  
);
```

```
CREATE TABLE STUDENT(  
  SID INT PRIMARY KEY,  
  Name VARCHAR(30) NOT NULL,  
  DOB DATE,  
  DeptID INT,  
  FOREIGN KEY(DeptID) REFERENCES DEPARTMENT(DeptID)  
);
```

Explanation

- PRIMARY KEY uniquely identifies each record.
- NOT NULL prevents empty values.
- FOREIGN KEY connects STUDENT with DEPARTMENT.

3. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

The query groups sales department-wise and finds departments whose average sales are greater than 1000.

```
SELECT DeptID,  
  COUNT(*) AS TotalSales,  
  SUM(Amount) AS TotalAmount,  
  AVG(Amount) AS AverageSales,  
  MAX(Amount) AS HighestSale,  
  MIN(Amount) AS LowestSale  
FROM Sales  
GROUP BY DeptID  
HAVING AVG(Amount) > 1000;
```

Explanation

- COUNT() counts records.
- SUM() calculates total sales.
- AVG() calculates average sales.
- MAX() finds highest sale.
- MIN() finds lowest sale.
- HAVING filters grouped data.

4. Write a correlated sub-query to find employees earning more than the average salary of their own department.

This query finds employees earning more than the average salary of their own department.

```
SELECT Name, Salary
FROM Employee E1
WHERE Salary >
(
  SELECT AVG(Salary)
  FROM Employee E2
  WHERE E1.DeptID=E2.DeptID
);
```

Explanation

- The inner query calculates department average salary.
- The outer query compares each employee's salary with that average.

5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables

INNER JOIN

Returns matching records from both tables.

```
SELECT *
FROM Employee
INNER JOIN Project
ON Employee.EmpID=Project.EmpID;
```

LEFT JOIN

Returns all employees and matching project details.

```
SELECT *
FROM Employee
LEFT JOIN Project
ON Employee.EmpID=Project.EmpID;
```

RIGHT JOIN

Returns all projects and matching employee details.

```
SELECT *
FROM Employee
RIGHT JOIN Project
ON Employee.EmpID=Project.EmpID;
```

FULL OUTER JOIN

Returns all records from both tables.

```
SELECT *
FROM Employee
FULL OUTER JOIN Project
ON Employee.EmpID=Project.EmpID;
```

CROSS JOIN

Returns every employee with every project.

```
SELECT *  
FROM Employee  
CROSS JOIN Project;
```

6. Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view. A view stores department-wise employee count and average salary.

```
CREATE VIEW DeptSalaryView AS  
SELECT DeptID,  
COUNT(*) AS TotalEmployees,  
AVG(Salary) AS AverageSalary  
FROM Employee  
GROUP BY DeptID;
```

Query the view:

```
SELECT * FROM DeptSalaryView;
```

Explanation

- A View is a virtual table.
- It stores the SQL query instead of data.
- It improves security and simplifies complex queries.

7. Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

Find the names of students enrolled in all courses.

π Name(Student $\bowtie$ Enrollment)

Explanation

- Join Student and Enrollment tables.
- Display only student names.
- This identifies students enrolled in all available courses.

8. Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

INSERT

```
INSERT INTO STUDENT  
VALUES(101,'Ravi','2005-06-10',1);
```

UPDATE

```
UPDATE STUDENT  
SET Name='Rahul'  
WHERE SID=101;
```

DELETE

```
DELETE FROM STUDENT  
WHERE SID=101;
```

Explanation

- INSERT adds a new record.
- UPDATE modifies existing data.
- DELETE removes unwanted records.
- Integrity constraints ensure only valid data is stored.

9. Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

Find employees working in the IT department and earning more than 50000.

{ E | Employee(E) AND E.Dept='IT' AND E.Salary>50000 }

Explanation

- Employee(E) represents employee tuples.
- Dept='IT' selects IT employees.
- Salary>50000 filters employees with higher salary.

10. Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

Relational Schema

- Passenger(PassengerID, Name, Phone)
- Flight(FlightID, FlightName, Source, Destination)
- Booking(BookingID, PassengerID, FlightID, JourneyDate)

SQL Queries

1. Join

```
SELECT Passenger.Name, Flight.FlightName
FROM Passenger
JOIN Booking
ON Passenger.PassengerID=Booking.PassengerID
JOIN Flight
ON Booking.FlightID=Flight.FlightID;
```

2. Subquery

```
SELECT Name
FROM Passenger
WHERE PassengerID IN
(SELECT PassengerID FROM Booking);
```

3. Create View

```
CREATE VIEW FlightView AS
SELECT FlightID, FlightName
FROM Flight;
```

4. Display View

```
SELECT * FROM FlightView;
```

5. Count Bookings

```
SELECT COUNT(*) AS TotalBookings
```

FROM Booking;

Explanation

- The schema stores passenger, flight, and booking details.
- Joins combine data from multiple tables.
- Subqueries retrieve records based on another query.
- Views simplify repeated queries.
- Aggregate functions summarize booking information.